

Resumen Ejecutivo – LogiTrack Escritorio

Autor: Jonathan Aldana

Fecha: Julio 2026

Versión: 1.0.0

1. Descripción del producto

LogiTrack Desktop es una aplicación de escritorio para el registro y seguimiento de envíos logísticos, desarrollada en Python 3.12 con Tkinter/ttk. Permite a operadores de sucursales registrar envíos, monitorear su estado en tiempo real, enriquecer registros con datos de geocodificación y clima de la ruta, y operar sin interrupciones aunque la conexión a internet sea intermitente.

El producto surgió de la necesidad de contar con una herramienta liviana, instalable en cualquier equipo Windows o macOS sin dependencias externas de plataforma, que centralice la información de envíos de cuatro sucursales (Central, Norte, Sur, Oriente) y ofrezca trazabilidad completa.

2. Decisiones clave y compensaciones

Framework de UI: Tkinter sobre PyQt / Kivy / Flet

Criterio	Tkinter	PyQt6	Flet
Licencia	Libre (stdlib)	GPL / comercial	Apache 2.0
Tamaño del ejecutable	~15 MB	~80 MB	~60 MB
Curva de aprendizaje	Baja	Alta	Medios de comunicación
Soporte nativo macOS/Windows	✓	✓	⚠ (Aleteo)

Decisión: Tkinter garantiza la distribución sin regalías y ejecutables compactos, aceptando la limitación de un sistema de estilos menos moderno.

Persistencia: SQLite sobre PostgreSQL / JSON

Se eligió SQLite 3 para ser un archivo único embebido en el ejecutable distribuido, con modo WAL para soportar lecturas simultáneas del hilo de UI y el trabajador en segundo plano sin bloqueos. No requiere

servidor ni configuración de red. El patrón repositorio abstracto (`EnvioRepository`) permite migrar a PostgreSQL en el futuro cambiando únicamente la implementación concreta, sin tocar servicios ni vistas.

Arquitectura MVC con capa de servicios específicos.

La separación estricta en cuatro capas (modelos → servicios → controladores → vistas) impone un costo inicial de estructura, pero elimina el acoplamiento que haría inviable la adición de nuevas vistas (web, CLI) o el reemplazo de componentes (repositorio, API de cliente). Las reglas de dependencia están verificadas automáticamente por pruebas AST en `tests/unit/test_arquitectura.py`.

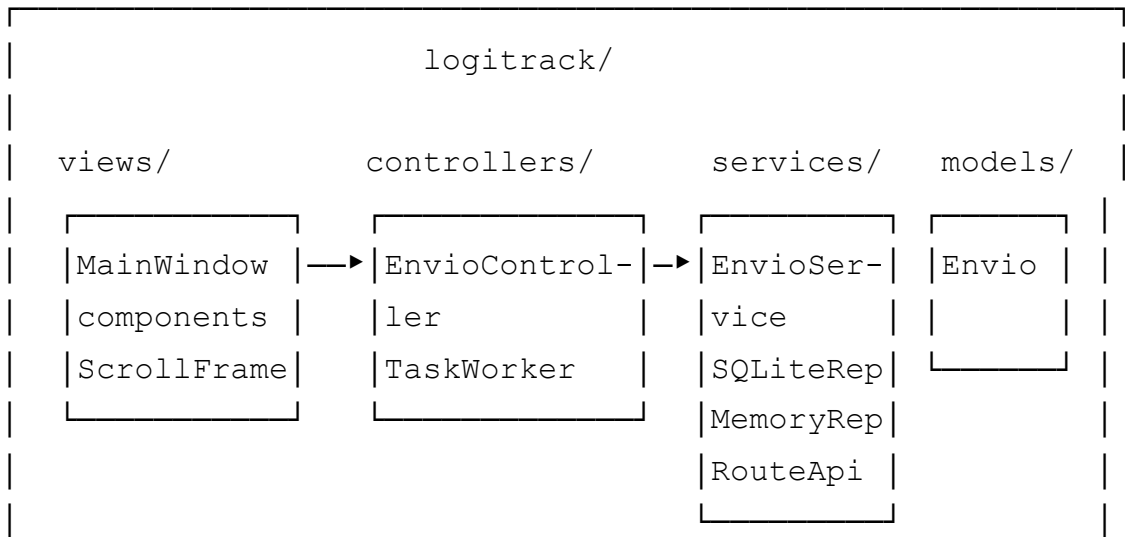
Asincronía: threading sobre asincronía

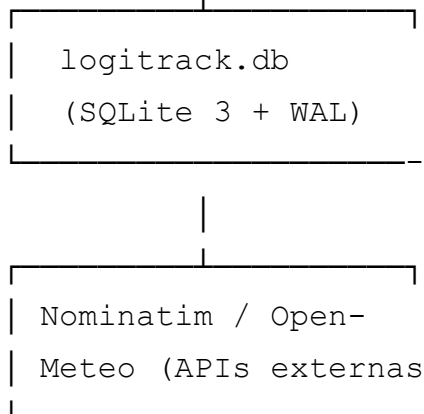
Las tareas lentas (carga desde BD, enriquecimiento con API) corren en `threading.Thread`. Se eligió threading en lugar de asincio porque Tkinter no es compatible con bucles de eventos de asincio en el hilo principal. El mecanismo de cancelación (`threading.Event`) y la devolución de resultados al hilo de UI (`root.after(0, callback)`) garantizan que la ventana nunca se congelará.

Cola sin conexión (`pending_ops`)

Cuando la API falla, la operación se persiste en la tabla `pending_ops` de SQLite. El botón "🔄 Sincronizar" reintenta todas las operaciones pendientes. Un error inicial causaba acumulación exponencial (cada sincronización re-encolaba las fallidas); se corrigió separando `encolar_si_falla=True/False` y deduplicando por `envio_id` al inicio de cada ciclo de sincronización.

3. Arquitectura del sistema





4. Hoja de ruta para producción real

Corto plazo (1-2 sprints)

Artículo	Justificación
Autenticación por usuario/rol	Actualmente cualquier persona con acceso al equipo puede modificar registros. Se requiere al menos autenticación básica con hash de contraseña (bcrypt).
Validación de direcciones en formulario	Nominatim acepta cualquier cadena; mostrar advertencia si la dirección no geocodifica antes de guardar.
Pruebas de integración con BD real	La carpeta <code>tests/integration/</code> está creada; Agregue pruebas que ejerciten <code>SQLiteRepository</code> contra un BD de prueba en memoria.

Mediano plazo (1 trimestre)

Artículo	Justificación
Soporte multisucursal con sincronización	Cada sucursal tiene su propia instancia; Se requiere un backend central (API REST o replicación SQLite) para consolidar datos.
Notificaciones de cambio de estado	Alertas al destinatario por correo electrónico/SMS cuando el estado cambia a "Entregado" o "Retrasado".
Dashboard de KPIs históricos	La tabla <code>logs</code> ya registra todas las operaciones; construir vistas analíticas de tiempos de entrega por sucursal.

Largo plazo

Artículo	Justificación
Actualizaciones automáticas	Integrar un mecanismo de actualización automática (Sparkle en macOS, NSIS en Windows) para distribuir nuevas versiones sin manual de intervención.
Migración a PostgreSQL	Cuando el volumen de envíos supera las capacidades de SQLite en modo local, migrar a PostgreSQL cambiando únicamente <code>SQLiteRepository</code> por <code>PostgreSQLRepository</code> (el patrón repositorio ya lo permite).
Versión web complementaria	La arquitectura MVC permite reutilizar servicios y modelos en una API FastAPI que alimenta una interfaz web para gerencia.

5. Métricas del proyecto

Métrica	Valor
Líneas de código Python	~1400
Archivos de código fuente	12
Pruebas automatizadas	5 (arquitectura + contratos de servicio)
Capas de arquitectura	4 (modelos, servicios, controladores, vistas)
Tablas SQLite	4 (envíos, sucursales, registros, operaciones pendientes)
API externas integradas	2 (Nominatim, Open-Meteo)
Tamaño del ejecutable (macOS)	~18 MB
Fases de desarrollo	8 iteraciones incrementales